MVLU COLLEGE
PRACTICAL NO 1

AIM:1.A Design and implement algorithms to encrypt and decrypt messages using classical substitution techniques

1. B Design and implement algorithms to encrypt and decrypt messages using transposition techniques

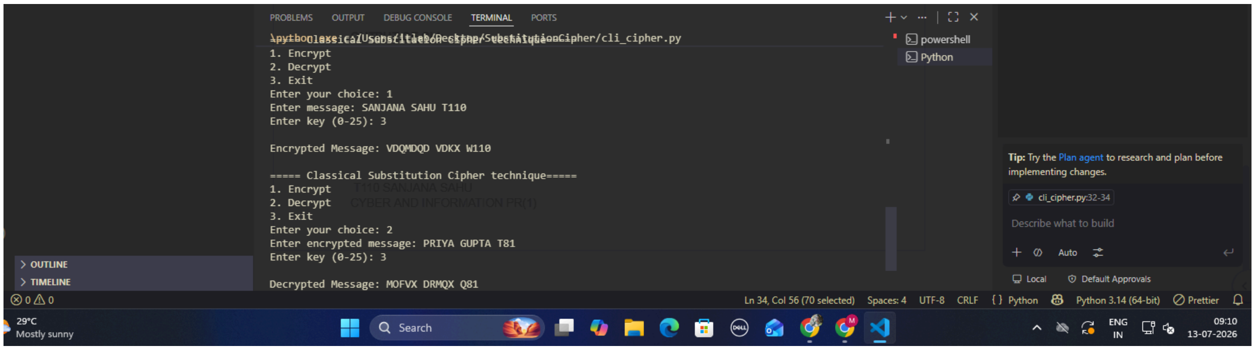
(A) CLAASICAL TECHNIQUES (Caesar CLI CODE)

```
# Caesar Cipher - CLI Version

def encrypt(text, key):
    result = ""

    for char in text:
        if char.isalpha():
            if char.isupper():
                result += chr((ord(char) - ord('A') + key) % 26 +
ord('A'))
            else:
                result += chr((ord(char) - ord('a') + key) % 26 +
ord('a'))
        else:
            result += char

    return result


def decrypt(text, key):
    result = ""

    for char in text:
        if char.isalpha():
            if char.isupper():
                result += chr((ord(char) - ord('A') - key) % 26 +
ord('A'))
            else:
                result += chr((ord(char) - ord('a') - key) % 26 +
ord('a'))
        else:
            result += char

    return result
```

```python
while True:
    print("\n===== Classical Substitution Cipher technique=====")
    print("1. Encrypt")
    print("2. Decrypt")
    print("3. Exit")

    choice = input("Enter your choice: ")

    if choice == "1":
        text = input("Enter message: ")
        key = int(input("Enter key (0-25): "))

        encrypted = encrypt(text, key)
        print("\nEncrypted Message:", encrypted)

    elif choice == "2":
        text = input("Enter encrypted message: ")
        key = int(input("Enter key (0-25): "))

        decrypted = decrypt(text, key)
        print("\nDecrypted Message:", decrypted)

    elif choice == "3":
        print("Program Exited.")
        break

    else:
        print("Invalid Choice! Try Again.")
```

OUTPUT OF CLI:

MVLU COLLEGE
PRACTICAL NO 1



(1)GUI OF CLAASICAL TECHNIQUES(Caesar)

```
import tkinter as tk
from tkinter import messagebox

# Encryption Function
def encrypt():
    text = entry_message.get()
    try:
        key = int(entry_key.get())
    except ValueError:
        messagebox.showerror("Error", "Enter a valid numeric key (0-25)")
        return

    result = ""

    for char in text:
        if char.isalpha():
            if char.isupper():
                result += chr((ord(char) - ord('A') + key) % 26 +
ord('A'))
            else:
                result += chr((ord(char) - ord('a') + key) % 26 +
ord('a'))
        else:
            result += char

    output.delete(1.0, tk.END)
    output.insert(tk.END, result)


# Decryption Function
def decrypt():
```

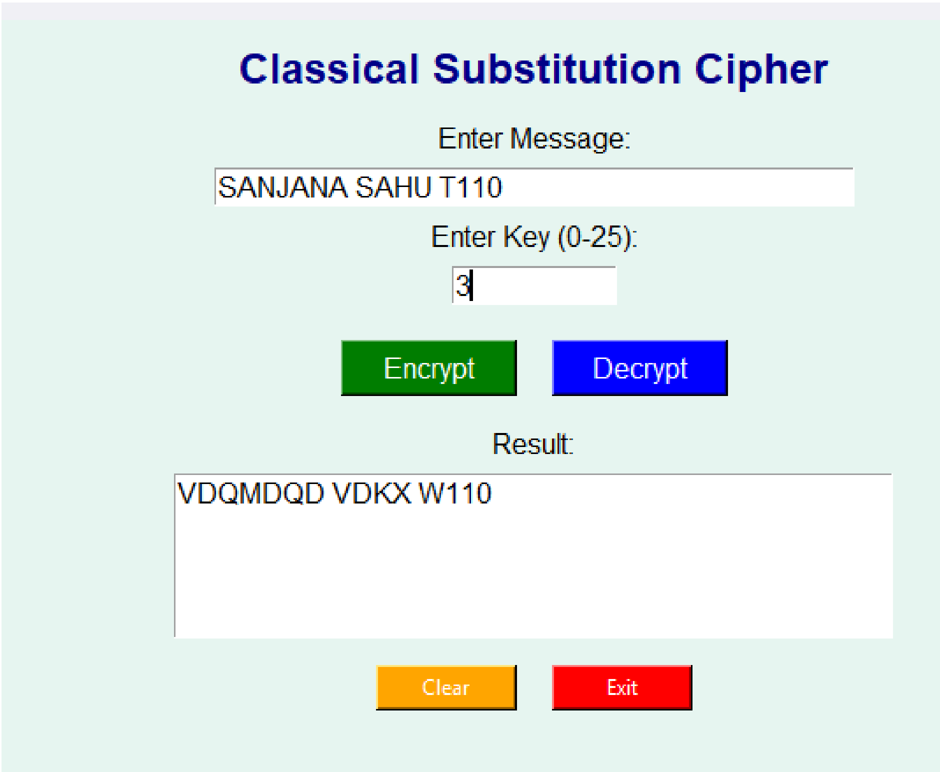
```
    text = entry_message.get()
    try:
        key = int(entry_key.get())
    except ValueError:
        messagebox.showerror("Error", "Enter a valid numeric key (0-25)")
        return

    result = ""

    for char in text:
        if char.isalpha():
            if char.isupper():
                result += chr((ord(char) - ord('A') - key) % 26 +
ord('A'))
            else:
                result += chr((ord(char) - ord('a') - key) % 26 +
ord('a'))
        else:
            result += char

    output.delete(1.0, tk.END)
    output.insert(tk.END, result)


# Clear Function
def clear():
    entry_message.delete(0, tk.END)
    entry_key.delete(0, tk.END)
    output.delete(1.0, tk.END)


# Main Window
root = tk.Tk()
root.title(" T110 Classical Substitution Cipher (Caesar Cipher)")
root.geometry("500x400")
root.configure(bg="#E8F6F3")

# Heading
title = tk.Label(root,
                 text="Classical Substitution Cipher",
```

```
                 font=("Arial", 18, "bold"),
                 bg="#E8F6F3",
                 fg="darkblue")
title.pack(pady=10)

# Message
tk.Label(root,
         text="Enter Message:",
         font=("Arial", 12),
         bg="#E8F6F3").pack()

entry_message = tk.Entry(root, width=40, font=("Arial", 12))
entry_message.pack(pady=5)

# Key
tk.Label(root,
         text="Enter Key (0-25):",
         font=("Arial", 12),
         bg="#E8F6F3").pack()

entry_key = tk.Entry(root, width=10, font=("Arial", 12))
entry_key.pack(pady=5)

# Buttons
frame = tk.Frame(root, bg="#E8F6F3")
frame.pack(pady=15)

btn_encrypt = tk.Button(frame,
                        text="Encrypt",
                        font=("Arial", 12),
                        bg="green",
                        fg="white",
                        width=10,
                        command=encrypt)

btn_encrypt.grid(row=0, column=0, padx=10)

btn_decrypt = tk.Button(frame,
                        text="Decrypt",
                        font=("Arial", 12),
```

```
                        bg="blue",
                        fg="white",
                        width=10,
                        command=decrypt)

btn_decrypt.grid(row=0, column=1, padx=10)

# Output
tk.Label(root,
         text="Result:",
         font=("Arial", 12),
         bg="#E8F6F3").pack()

output = tk.Text(root,
                 height=5,
                 width=45,
                 font=("Arial", 12))
output.pack(pady=5)

# Bottom Buttons
frame2 = tk.Frame(root, bg="#E8F6F3")
frame2.pack(pady=10)

btn_clear = tk.Button(frame2,
                      text="Clear",
                      width=10,
                      bg="orange",
                      fg="white",
                      command=clear)

btn_clear.grid(row=0, column=0, padx=10)

btn_exit = tk.Button(frame2,
                     text="Exit",
                     width=10,
                     bg="red",
                     fg="white",
                     command=root.destroy)

btn_exit.grid(row=0, column=1, padx=10)
```

```
root.mainloop()
```

OUTPUT:

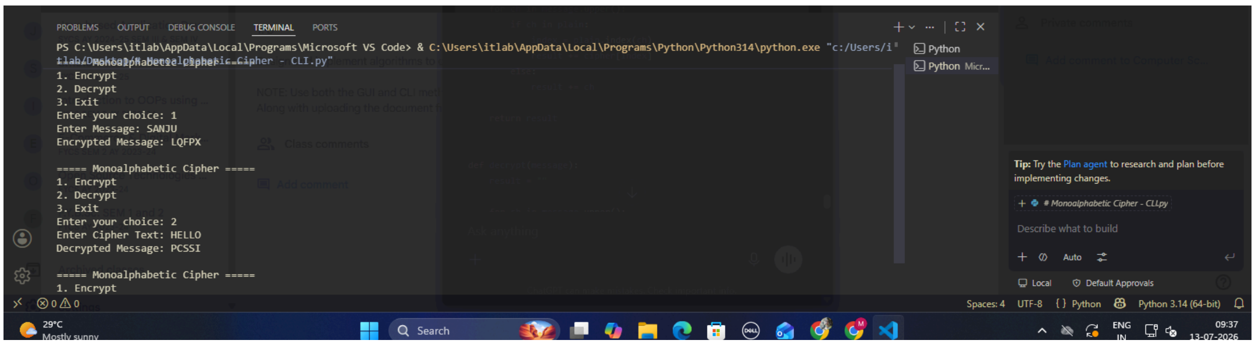


(2)Monoalphabetic Cipher(CLI)

```
# Monoalphabetic Cipher - CLI

plain = "ABCDEFGHIJKLMNOPQRSTUVWXYZ"
cipher = "QWERTYUIOPASDFGHJKLZXCVBNM"
```

```python
def encrypt(message):
    result = ""

    for ch in message.upper():
        if ch in plain:
            index = plain.index(ch)
            result += cipher[index]
        else:
            result += ch

    return result


def decrypt(message):
    result = ""

    for ch in message.upper():
        if ch in cipher:
            index = cipher.index(ch)
            result += plain[index]
        else:
            result += ch

    return result


while True:
    print("\n===== Monoalphabetic Cipher =====")
    print("1. Encrypt")
    print("2. Decrypt")
    print("3. Exit")

    choice = input("Enter your choice: ")

    if choice == "1":
        msg = input("Enter Message: ")
        print("Encrypted Message:", encrypt(msg))
```

```python
    elif choice == "2":
        msg = input("Enter Cipher Text: ")
        print("Decrypted Message:", decrypt(msg))

    elif choice == "3":
        print("Program Closed.")
        break

    else:
        print("Invalid Choice!")
```

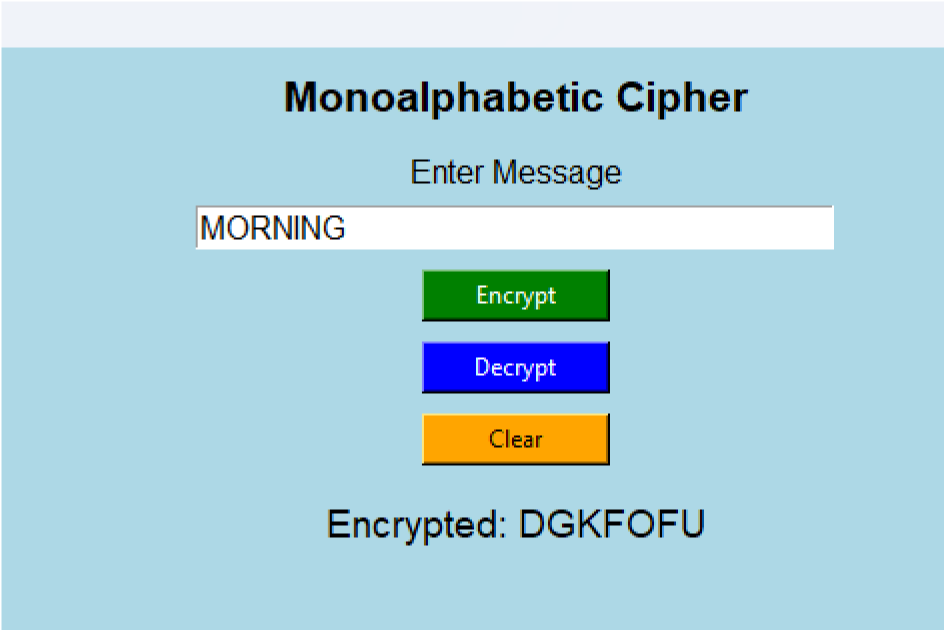


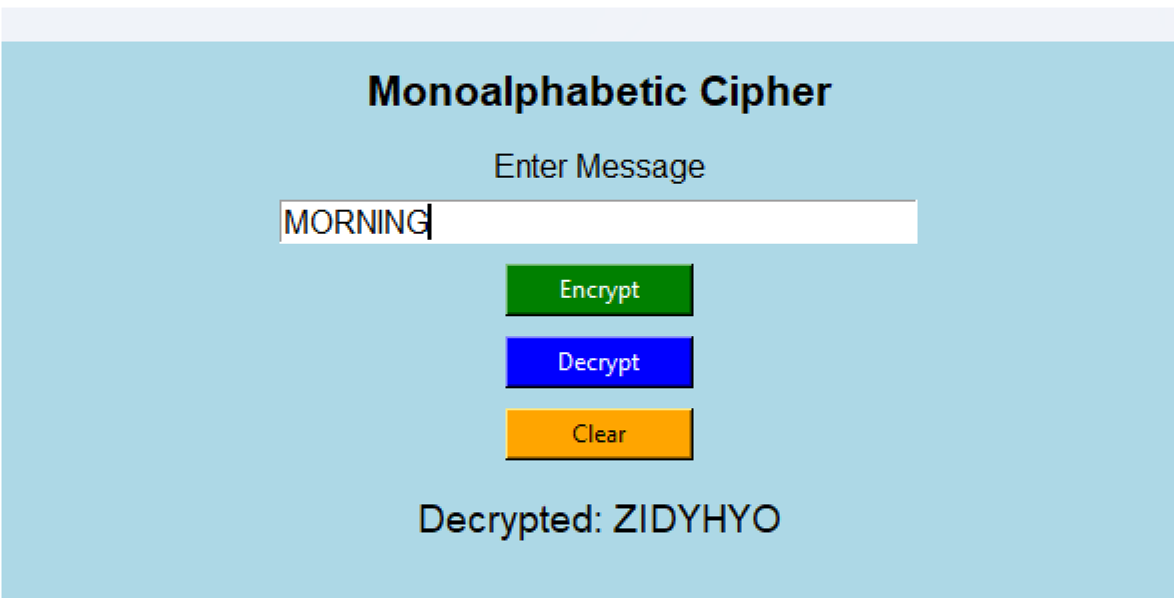
2. GUI

```python
import tkinter as tk
from tkinter import messagebox

plain = "ABCDEFGHIJKLMNOPQRSTUVWXYZ"
cipher = "QWERTYUIOPASDFGHJKLZXCVBNM"


def encrypt():
    msg = message_entry.get().upper()

    result = ""

    for ch in msg:
        if ch in plain:
            result += cipher[plain.index(ch)]
        else:
            result += ch

    result_label.config(text="Encrypted: " + result)
```

```
def decrypt():
    msg = message_entry.get().upper()

    result = ""

    for ch in msg:
        if ch in cipher:
            result += plain[cipher.index(ch)]
        else:
            result += ch

    result_label.config(text="Decrypted: " + result)


def clear():
    message_entry.delete(0, tk.END)
    result_label.config(text="")


root = tk.Tk()
root.title("Monoalphabetic Cipher")
root.geometry("450x300")
root.configure(bg="lightblue")

title = tk.Label(root,
                 text="Monoalphabetic Cipher",
                 font=("Arial",16,"bold"),
                 bg="lightblue")
title.pack(pady=10)

tk.Label(root,
         text="Enter Message",
         bg="lightblue",
         font=("Arial",12)).pack()

message_entry = tk.Entry(root,
                         width=35,
                         font=("Arial",12))
message_entry.pack(pady=5)
```

```
tk.Button(root,
          text="Encrypt",
          command=encrypt,
          bg="green",
          fg="white",
          width=12).pack(pady=5)

tk.Button(root,
          text="Decrypt",
          command=decrypt,
          bg="blue",
          fg="white",
          width=12).pack(pady=5)

tk.Button(root,
          text="Clear",
          command=clear,
          bg="orange",
          width=12).pack(pady=5)

result_label = tk.Label(root,
                        text="",
                        bg="lightblue",
                        font=("Arial",14))
result_label.pack(pady=10)

root.mainloop()
```





Transposition techniques

1. Rail Fence Cipher (CLI)

```
def encrypt(text, key):
    rail = [['\n' for i in range(len(text))]
            for j in range(key)]

    dir_down = False
    row, col = 0, 0

    for ch in text:
```

```
        if row == 0 or row == key - 1:
            dir_down = not dir_down

        rail[row][col] = ch
        col += 1

        if dir_down:
            row += 1
        else:
            row -= 1

    result = ""
    for i in range(key):
        for j in range(len(text)):
            if rail[i][j] != '\n':
                result += rail[i][j]
    return result


def decrypt(cipher, key):
    rail = [['\n' for i in range(len(cipher))]
            for j in range(key)]

    dir_down = None
    row, col = 0, 0

    for i in range(len(cipher)):
        if row == 0:
            dir_down = True
        if row == key - 1:
            dir_down = False

        rail[row][col] = '*'
        col += 1

        if dir_down:
            row += 1
        else:
            row -= 1

    index = 0
    for i in range(key):
        for j in range(len(cipher)):
            if rail[i][j] == '*' and index < len(cipher):
```

```
                rail[i][j] = cipher[index]
                index += 1

    result = ""
    row, col = 0, 0

    for i in range(len(cipher)):
        if row == 0:
            dir_down = True
        if row == key - 1:
            dir_down = False

        result += rail[row][col]
        col += 1

        if dir_down:
            row += 1
        else:
            row -= 1

    return result


while True:
    print("\n===== Rail Fence Cipher =====")
    print("1. Encrypt")
    print("2. Decrypt")
    print("3. Exit")

    ch = input("Enter Choice: ")

    if ch == '1':
        text = input("Enter Message: ")
        key = int(input("Enter Key: "))
        print("Encrypted:", encrypt(text, key))

    elif ch == '2':
        text = input("Enter Cipher Text: ")
        key = int(input("Enter Key: "))
        print("Decrypted:", decrypt(text, key))

    elif ch == '3':
        break
```

```
else:
    print("Invalid Choice")
```

```
PS C:\Users\itlab\Desktop\TRASPORTATION TEHNIQUE> & C:\Users\itlab\AppData\Local\Programs\Python\Python314\python.e
xe "c:/Users/itlab/Desktop/TRASPORTATION TEHNIQUE/rail_fence_cipher_cli.py"
2. Decrypt
3. Exit
Enter Choice: 1
Enter Message: T110
Enter Key: 3
Encrypted: T101

===== Rail Fence Cipher =====
1. Encrypt
2. Decrypt
3. Exit
Enter Choice: 2
Enter Cipher Text: T110
Enter Key: 3
Decrypted: T101
```

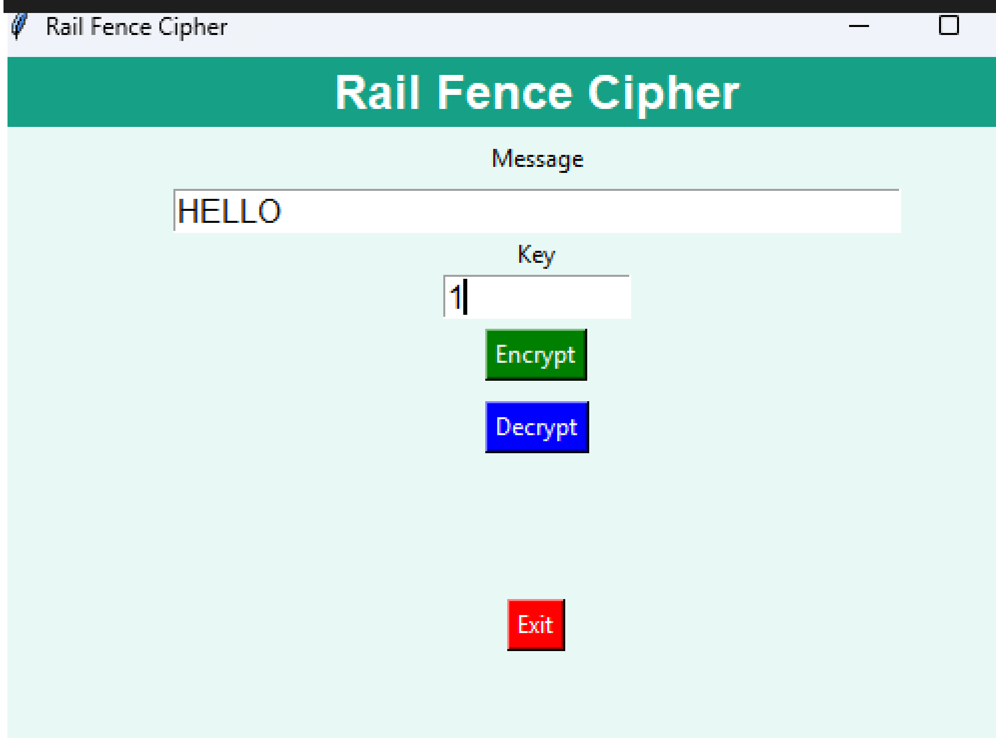
1. Rail Fence Cipher (GUI)

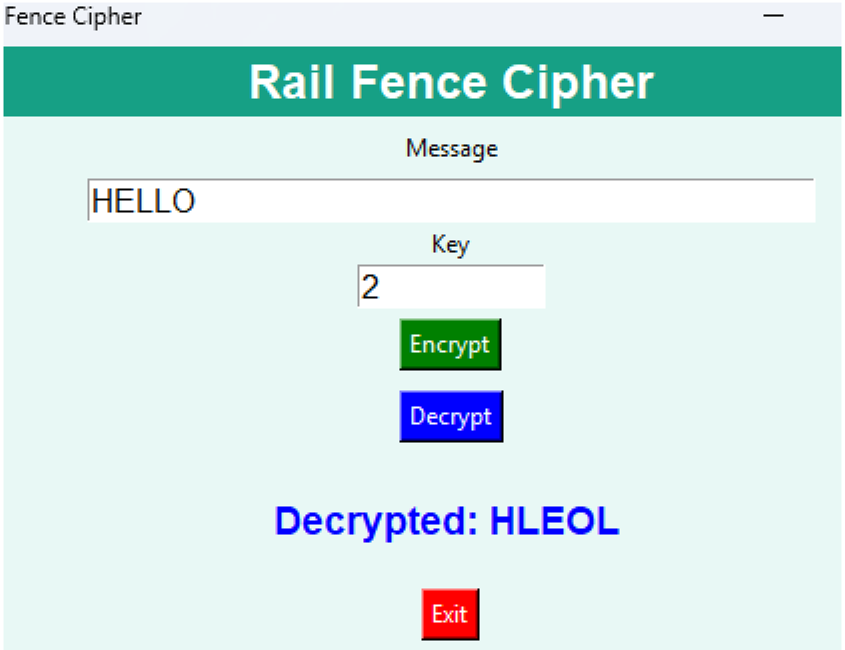
```
import tkinter as tk
from tkinter import messagebox

def encrypt(text, key):
    rail = [['\n' for i in range(len(text))]
            for j in range(key)]

    dir_down = False
    row, col = 0, 0

    for ch in text:
        if row == 0 or row == key - 1:
            dir_down = not dir_down

        rail[row][col] = ch
        col += 1

        if dir_down:
            row += 1
        else:
            row -= 1

    result = ""
    for i in range(key):
        for j in range(len(text)):
            if rail[i][j] != '\n':
                result += rail[i][j]
```

```
    return result


def decrypt(cipher, key):
    rail = [['\n' for i in range(len(cipher))]
            for j in range(key)]

    dir_down = None
    row, col = 0, 0

    for i in range(len(cipher)):
        if row == 0:
            dir_down = True
        if row == key - 1:
            dir_down = False

        rail[row][col] = '*'
        col += 1

        if dir_down:
            row += 1
        else:
            row -= 1

    index = 0
    for i in range(key):
        for j in range(len(cipher)):
            if rail[i][j] == '*' and index < len(cipher):
                rail[i][j] = cipher[index]
                index += 1

    result = ""
    row, col = 0, 0

    for i in range(len(cipher)):
        if row == 0:
            dir_down = True
        if row == key - 1:
            dir_down = False
```

```
        result += rail[row][col]
        col += 1

        if dir_down:
            row += 1
        else:
            row -= 1

    return result


def enc():
    result.config(text="Encrypted: " +
                  encrypt(msg.get(), int(key.get())),
                  fg="green")


def dec():
    result.config(text="Decrypted: " +
                  decrypt(msg.get(), int(key.get())),
                  fg="blue")


root = tk.Tk()
root.title("Rail Fence Cipher")
root.geometry("550x350")
root.configure(bg="#E8F8F5")

tk.Label(root,
         text="Rail Fence Cipher",
         bg="#16A085",
         fg="white",
         font=("Arial",18,"bold")).pack(fill="x")

tk.Label(root,text="Message",bg="#E8F8F5").pack(pady=5)
msg = tk.Entry(root,width=40,font=("Arial",12))
msg.pack()

tk.Label(root,text="Key",bg="#E8F8F5").pack()
key = tk.Entry(root,width=10,font=("Arial",12))
```

```
key.pack()

tk.Button(root,text="Encrypt",
          bg="green",fg="white",
          command=enc).pack(pady=5)

tk.Button(root,text="Decrypt",
          bg="blue",fg="white",
          command=dec).pack(pady=5)

result = tk.Label(root,text="",bg="#E8F8F5",
                  font=("Arial",14,"bold"))
result.pack(pady=20)

tk.Button(root,text="Exit",
          bg="red",fg="white",
          command=root.destroy).pack()

root.mainloop()
```



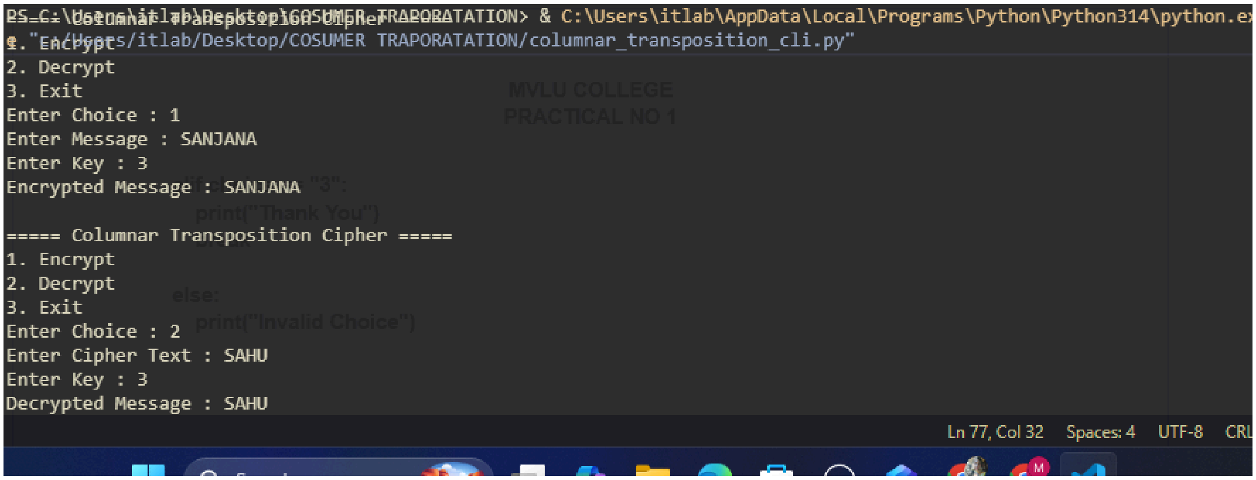


(2)Columnar Transposition Cipher (CLI)

```
def encrypt(message, key):
    col = len(key)
    row = (len(message) + col - 1) // col

    message += 'X' * (row * col - len(message))

    matrix = []
    index = 0

    for i in range(row):
        matrix.append(list(message[index:index + col]))
        index += col

    order = sorted(list(enumerate(key)), key=lambda x: x[1])

    cipher = ""

    for pos, ch in order:
        for r in range(row):
            cipher += matrix[r][pos]

    return cipher
```

```
def decrypt(cipher, key):
    col = len(key)
    row = len(cipher) // col

    order = sorted(list(enumerate(key)), key=lambda x: x[1])

    matrix = [['' for _ in range(col)] for _ in range(row)]

    index = 0

    for pos, ch in order:
        for r in range(row):
            matrix[r][pos] = cipher[index]
            index += 1

    plain = ""

    for r in range(row):
        for c in range(col):
            plain += matrix[r][c]

    return plain.rstrip('X')


while True:

    print("\n===== Columnar Transposition Cipher =====")
    print("1. Encrypt")
    print("2. Decrypt")
    print("3. Exit")

    choice = input("Enter Choice : ")

    if choice == "1":
        text = input("Enter Message : ").upper()
        key = input("Enter Key : ").upper()

        print("Encrypted Message :", encrypt(text, key))

    elif choice == "2":
        text = input("Enter Cipher Text : ").upper()
        key = input("Enter Key : ").upper()

        print("Decrypted Message :", decrypt(text, key))
```

```
    elif choice == "3":
        print("Thank You")
        break

    else:
        print("Invalid Choice")
```



Columnar Transposition Cipher (GUI)

```
import tkinter as tk
from tkinter import messagebox


def encrypt(message, key):
    col = len(key)
    row = (len(message) + col - 1) // col

    message += 'X' * (row * col - len(message))

    matrix = []
    index = 0

    for i in range(row):
        matrix.append(list(message[index:index + col]))
        index += col

    order = sorted(list(enumerate(key)), key=lambda x: x[1])

    cipher = ""
```

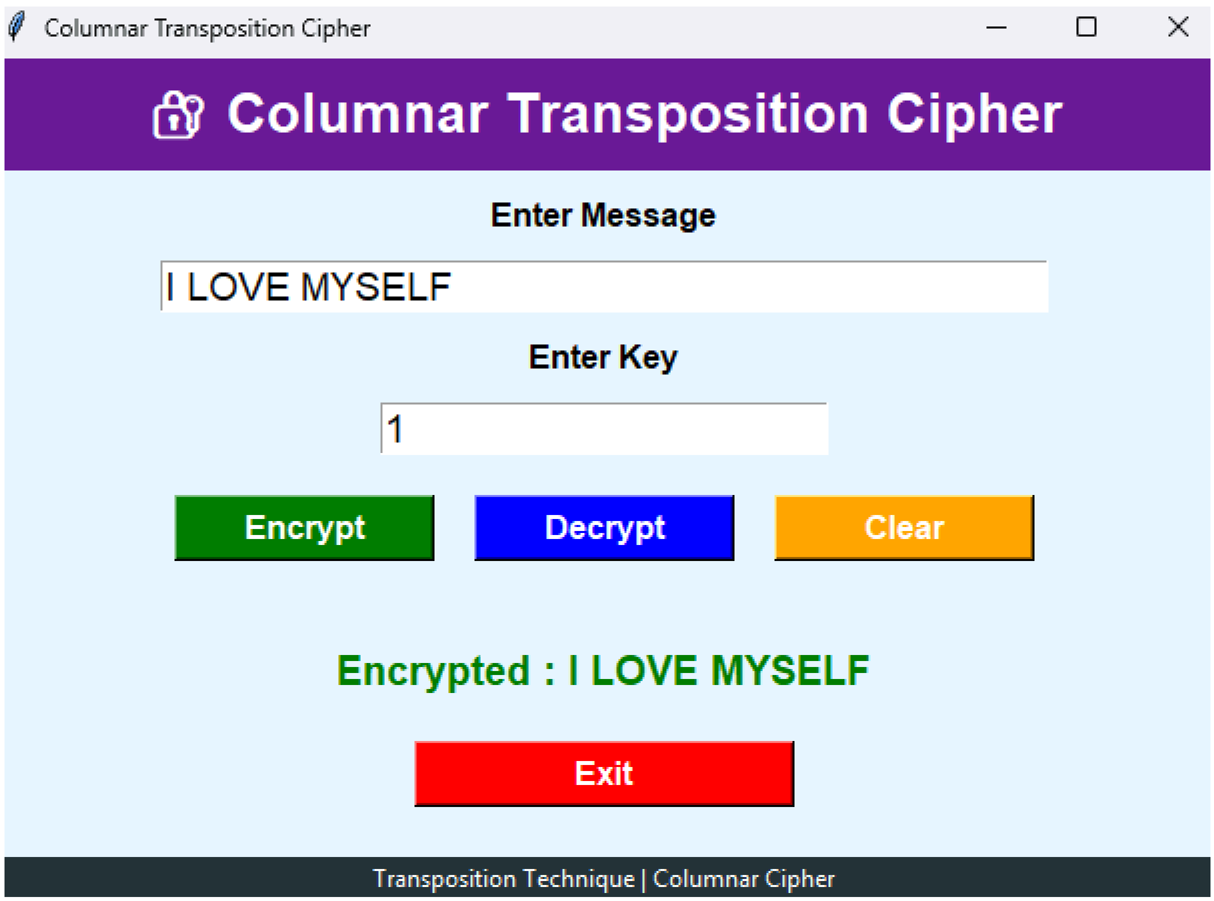
```
    for pos, ch in order:
        for r in range(row):
            cipher += matrix[r][pos]

    return cipher


def decrypt(cipher, key):
    col = len(key)
    row = len(cipher) // col

    order = sorted(list(enumerate(key)), key=lambda x: x[1])

    matrix = [['' for _ in range(col)] for _ in range(row)]

    index = 0

    for pos, ch in order:
        for r in range(row):
            matrix[r][pos] = cipher[index]
            index += 1

    plain = ""

    for r in range(row):
        for c in range(col):
            plain += matrix[r][c]

    return plain.rstrip('X')


def do_encrypt():
    msg = msg_entry.get().upper()
    key = key_entry.get().upper()

    if msg == "" or key == "":
        messagebox.showerror("Error", "Please enter Message and Key")
        return

    result.config(text="Encrypted : " + encrypt(msg, key),
                  fg="green")


def do_decrypt():
```

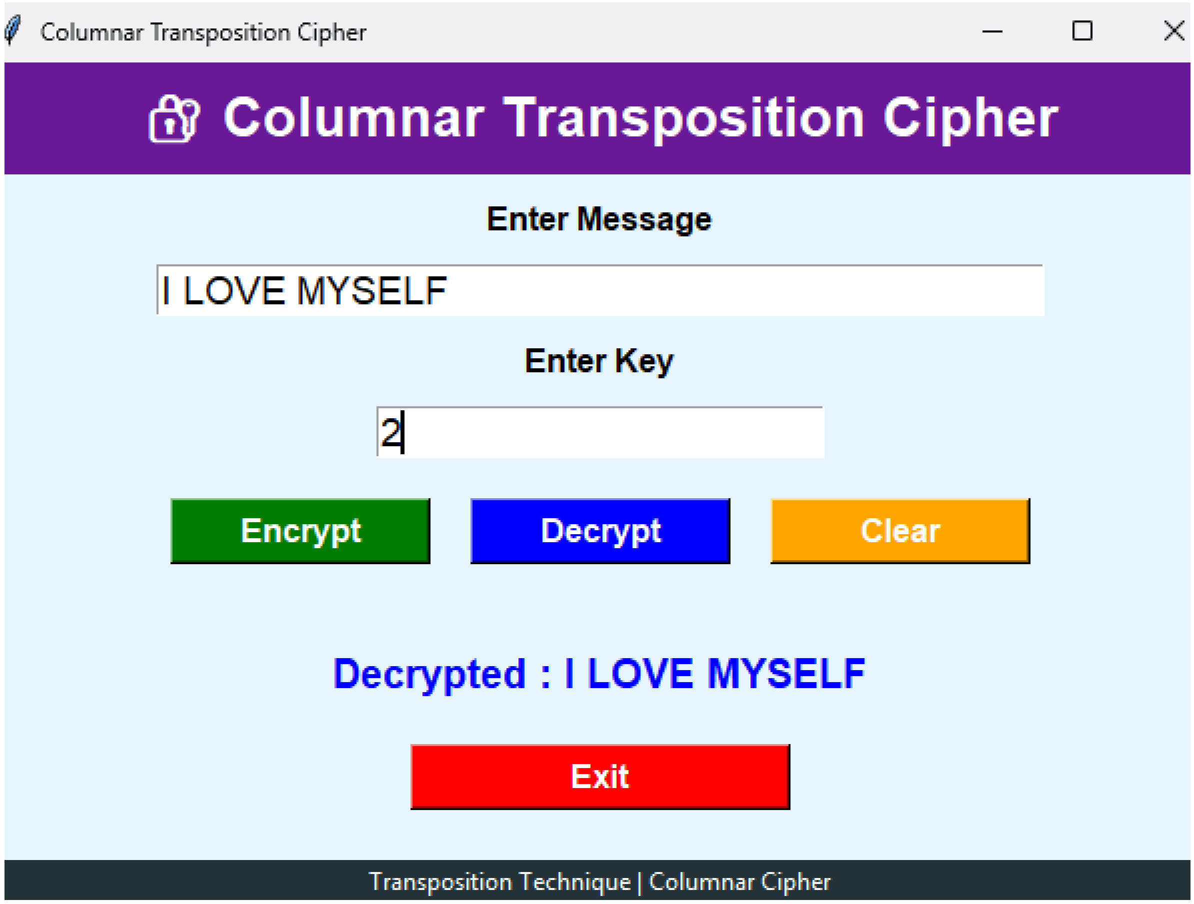
```
    msg = msg_entry.get().upper()
    key = key_entry.get().upper()

    if msg == "" or key == "":
        messagebox.showerror("Error", "Please enter Message and Key")
        return

    result.config(text="Decrypted : " + decrypt(msg, key),
                  fg="blue")


def clear():
    msg_entry.delete(0, tk.END)
    key_entry.delete(0, tk.END)
    result.config(text="")


root = tk.Tk()
root.title("Columnar Transposition Cipher")
root.geometry("620x420")
root.configure(bg="#EAF8FF")

header = tk.Label(root,
                  text="🔐 Columnar Transposition Cipher",
                  bg="#6A1B9A",
                  fg="white",
                  font=("Arial",20,"bold"),
                  pady=10)
header.pack(fill="x")

tk.Label(root,
         text="Enter Message",
         bg="#EAF8FF",
         font=("Arial",12,"bold")).pack(pady=10)

msg_entry = tk.Entry(root,
                     width=40,
                     font=("Arial",14))
msg_entry.pack()

tk.Label(root,
         text="Enter Key",
         bg="#EAF8FF",
         font=("Arial",12,"bold")).pack(pady=10)
```

```
key_entry = tk.Entry(root,
                width=20,
                font=("Arial",14))
key_entry.pack()

frame = tk.Frame(root,bg="#EAF8FF")
frame.pack(pady=20)

tk.Button(frame,
        text="Encrypt",
        bg="green",
        fg="white",
        font=("Arial",12,"bold"),
        width=12,
        command=do_encrypt).grid(row=0,column=0,padx=10)

tk.Button(frame,
        text="Decrypt",
        bg="blue",
        fg="white",
        font=("Arial",12,"bold"),
        width=12,
        command=do_decrypt).grid(row=0,column=1,padx=10)

tk.Button(frame,
        text="Clear",
        bg="orange",
        fg="white",
        font=("Arial",12,"bold"),
        width=12,
        command=clear).grid(row=0,column=2,padx=10)

result = tk.Label(root,
              text="",
              bg="#EAF8FF",
              font=("Arial",15,"bold"))
result.pack(pady=20)

tk.Button(root,
        text="Exit",
        bg="red",
        fg="white",
        font=("Arial",12,"bold"),
```

```
        width=18,
        command=root.destroy).pack()

footer = tk.Label(root,
              text="Transposition Technique | Columnar Cipher",
              bg="#263238",
              fg="white")
footer.pack(side="bottom",fill="x")

root.mainloop()
```

OUTPUT:




Columnar Transposition Cipher
Columnar Transposition Cipher
Enter Message
I LOVE MYSELF
Enter Key
2
Encrypt
Decrypt
Clear
Decrypted : I LOVE MYSELF
Exit
Transposition Technique | Columnar Cipher